Evaluating SYNGrid: A Configurable Benchmark Environment for Temporal Credit Assignment in Reinforcement Learning

Joel Lansgren
Department of Communication, Quality Management and Information Systems
Mid Sweden University
Östersund, Sweden
jola2311@student.miun.se

***Abstract*—A central challenge in Artificial Intelligence (AI) is assigning credit to the correct actions within long chains of events that eventually lead to a reward, something Minsky noted back in 1961. This is the heart of the Credit Assignment Problem (CAP), and targeted benchmarks that isolate it from exploration challenges are still lacking.**

The goal of this thesis is to evaluate SYNGrid, a configurable grid-based AI benchmark framework, focusing on its tiered orb mechanic and configurable delay as controlled measures of Temporal Credit Assignment (TCA).

Three types of agents are compared to assess if SYNGrid produces meaningful differentiation across architectures. The agents include a stateless baseline, a frame-stacking agent, and a Long Short term Memory (LSTM)-based agent sharing the same underlying algorithm. TCA is assessed by comparing agent performance under three distinct scenarios in the environment.

The benchmark design is also evaluated for its ability to produce distinct and meaningful task variations without code changes.

Results show that the stateless agent outperformed both memory-augmented agents across most scenarios, with LSTMs advantage emerging primarily under partial observability where its ability to reconstruct spatial context proved decisive, however, higher explained variance and lower entropy across all scenarios suggest a more structured understanding of the task compared to the other two. Frame stacking offered marginal benefit in the same task and degraded performance elsewhere. These distinctions were made possible through configuration alone, though a sprawling config file places cognitive load on the user that a dedicated interface could reduce.

SYNGrid produces measurable differentiation across TCA challenges, partially isolating it from exploration and perception across three evaluated scenarios. Entanglement is possible, as the spatial scenario illustrates, but reflects the framework's expressive range rather than a fundamental limitation. Scenario construction through configuration alone proved functional and flexible, at the cost of cognitive overhead that a dedicated interface could address.

***Index Terms*—Artificial Intelligence, Temporal Credit Assignment, Reinforcement Learning, Gymnasium, Design Science**

I. Definitions

LTCA	Long-Term Credit Assignment
DRL	Deep Reinforcement Learning
RL	Reinforcement Learning
GUI	Graphical User Interface
LLM	Large Language Model
AI	Artificial Intelligence
MDP	Markovian Decision Process
POMDP	Partially Observable Markov Decision Process
CAP	Credit Assignment Problem
CA	Credit Assignment
TCAP	Temporal Credit Assignment Problem
TCA	Temporal Credit Assignment
NLE	NetHack Learning Environment
LSTM	Long Short term Memory
SB3	Stable-Baselines3
PPO	Proximal Policy Optimization
FSPPO	Frame Stacking Proximal Policy Optimization
RPPO	Recurrent Proximal Policy Optimization
NN	Neural Networks
GAE	Generalized Advantage Estimation
CSV	Comma Separated Values
DSR	Design Science Research
YAML	YAML Ain't Markup Language

Acknowledgment

I want to thank my wife for her patience during the late nights this thesis demanded, and for listening without complaint whenever I needed to talk it through — even when the details made no sense to her. I also want to thank my father for selflessly taking care of our dog so I could have the focus this final stretch demanded. Big thanks also to my supervisor Rodi Jolak who ensured I kept my schedule and helped keep the text clean and readable, which is hard-won advice from someone who knows how these things tend to grow. And lastly, thanks to the AI assistants who helped me think through problems and refine ideas during this project,

serving as a valuable resource for a distance student working alone.

II. Introduction

TCA, the problem of evaluating the utility of individual actions within a long sequence, remains a long-standing challenge for modern AI agents. As Minsky concisely put it back in 1961:

> In playing a complex game such as chess or checkers, or in writing a computer program, one has a definite success criterion — the game is won or lost. But in the course of play, each ultimate success (or failure) is associated with a vast number of internal decisions. If the run is successful, how can we assign credit for the success among the multitude of decisions? [1, p. 20]

Even in 2019, memory-enabled Reinforcement Learning (RL) approaches showed persistent difficulties with Long-Term Credit Assignment (LTCA), and a 2024 survey confirmed that the problem persists even among state-of-the-art methods [2], [3]. This is partly by design, since early Deep Reinforcement Learning (DRL) research deliberately avoided tasks where Credit Assignment (CA) was hard, as doing so would have got in the way of more basic problems that needed solving first. With those now largely addressed, the field is shifting from expansion toward refinement, and problems like TCA are coming back into focus [3].

While understanding both TCA and LTCA is critical, translating these insights into practical evaluation is non-trivial: most existing benchmarks capable of properly measuring this entangle multiple challenges like exploration, perception, and planning, making it difficult to isolate failure modes. They also often require computational resources beyond the reach of typical labs [4, sec. 3.2.3] [5]. More controlled environments provide multiple distinct tasks that are useful for probing different aspects of TCA [3, sec. 7.2.1]. However, they are often static or require extensive framework knowledge to create custom tasks.

To address these gaps, there is a need for a flexible and complex environment that remains lightweight compared to larger-scale benchmarks, while preserving the benefits of controlled tasks and lowering the barrier to task design. SYNGrid intends to meet those needs by being configurable: task complexity scales along interpretable axes like tier depth, delay length or active orb count, while targeted scenarios can be constructed by selecting a specific orb mix, all through a single configuration file without code changes. A human-playable mode is also provided, allowing scenario balance to be validated before committing to a training run. Of these, the tier mechanic together with a configurable delay serves as the primary evaluation axis for this thesis.

The aim of this thesis is to evaluate SYNGrid in its current form as a proof of concept, using the design science approach recommended by SIGSOFT[1]. The evaluation addresses two dimensions: its effectiveness at measuring TCA across AI agents, and whether its configuration-driven design produces distinguishable scenarios without source-code changes.

III. Purpose and Contributions

As foundational problems in DRL have largely been addressed, attention is returning to harder challenges like CA, and benchmarks that systematically isolate CAP are becoming more relevant. Yet, current ones tend to entangle it in other challenges. This makes it difficult to isolate and measure an agent's ability to assign credit over long sequences. Many of them also require significant computational resources, or lack configurability for easy creation of custom test scenarios [3].

The problem is that without lightweight, configurable benchmarks that isolates TCA, the field lacks shared, low-cost places to systematically stress-test how different architectures handle long-term dependencies.

Therefore, the goal of this thesis is to evaluate SYNGrid as a lightweight benchmark for TCA. In focus is its tier orb mechanic, which isolates sequential CA as a single tunable axis of difficulty, and its configuration-driven design, which allows new scenarios to emerge from toggling mechanics rather than rewriting code. The tier orb is one piece of a larger combinable system that the SYN in SYNGrid points toward; this thesis evaluates that first piece.

To guide this evaluation, two research questions are formulated:

- RQ1: To what extent can SYNGrid reliably measure TCA across different agent architectures while isolating it from other challenges through natural resource mechanics?
- RQ2: To what extent does SYNGrid's configuration-driven design produce learning outcomes consistent with the intended difficulty manipulations, without modifying the underlying source code?

Answering these questions requires not only empirical agent experiments but also a functioning, extensible benchmark implementation. The technical contribution of this thesis is therefore twofold: first, the SYNGrid framework itself; second, its empirical evaluation as a TCA measurement tool.

IV. Background

This section introduces the concepts and terminology necessary to follow the experiments and results in this thesis. It covers benchmarking in RL and why isolation matters, the RL paradigm, the CA problem and its longer-term variants,

[1] https://www2.sigsoft.org/EmpiricalStandards/docs/standards?standard=EngineeringResearch

the observability conditions under which agents are evaluated, and the algorithms used.

A. Benchmarks in Reinforcement Learning

A benchmark in RL is like a benchmark in any engineering discipline. However, a benchmark can only tell you what it isolates [3]. Take mining diamonds in Minecraft as an example: sequential decision-making is clearly involved, but so is exploration, inventory management, and combat. If an agent fails, it's hard to tell whether it struggled to plan ahead, failed to find the right resources, or simply never encountered the right situation. Introduce a map to the agent, and performance improves. But was CA the bottleneck, or was it exploration? The benchmark alone cannot say. Isolating the cause requires additional experiments, and even then the answer isn't clean. A benchmark designed around isolation doesn't eliminate that uncertainty, but it has the capability to turn a scoreboard into a diagnostic tool.

B. Reinforcement Learning Basics

RL is not a specific algorithm, the model itself or a model architecture, but a learning paradigm. It's a way of training models that can be implemented across many different model architectures, from simple Neural Networkss (NNs) to recurrent models and Large Language Models (LLMs) [6]. Take a dog learning a sequence: push a lever, open a hatch, retrieve the treat. The dog observes, acts, and adjusts based on what worked. Formalized, this is the RL loop: state, action, reward, new state, repeated until behavior improves.

What the agent is building through this process is a policy — its learned strategy for deciding what to do given what it currently observes. At the start of training the policy is random. Over time, if the reward signal is informative enough, it improves.

In SYNGrid the environment is episodic. That means to say that each episode begins with the agent placed in the grid and ends when a terminal condition is reached. Either through a tier chain break, a completed chain, scores reduced to zero or the step budget expiring. All depending on what kind of task you are running. Learning happens across thousands of episodes, with the policy gradually improving as the agent accumulates experience.

C. Credit Assignment

When an agent reaches a goal after a sequence of actions, how does it know which actions actually mattered? This is the CAP — the challenge of distributing credit for an outcome back across the decisions that led to it [3]. In a short sequence the answer is relatively straightforward. In longer ones it becomes a central problem in RL.

The temporal dimension makes CAP harder. When a reward arrives long after the actions that caused it, the learning signal has to travel back across many timesteps to reach the decisions that actually mattered, and this narrower problem is known as the Temporal Credit Assignment Problem (TCAP) [3]. The further back that signal has to reach, the weaker and noisier it becomes.

LTCA is the specific case where the temporal gap is long enough that reliable learning breaks down. [2]

Beyond SYNGrid, RL is being applied to an expanding set of real-world problems: robotics, energy management, autonomous driving and even stratospheric balloon navigation, to name a few. What these share is scale where many decisions unfolds over long horizons, where success depends on getting a long chain of them right. Solving the CAP and learning which of those decisions really mattered, is what lets an agent reliably improve rather than merely repeat what happened to work. And this gets harder as environments scale toward real-world complexity. As they do, each action accounts for a vanishing fraction of the outcome, and the signal needed to tell a decisive action apart from an irrelevant one grows fainter. Solving the task by finding one winning policy is a narrower achievement than understanding why it won. The deeper knowledge of which actions were truly decisive is also the kind that could carry over, because the world these tasks share operates on the same underlying rules, credit learned in one setting can become a lever in another [3, sec. 4.0].

On top of this difficulty, in the sequential dependency tasks that are an inherent factor of the tier mechanic used in SYNGrid, the reward landscape adds its own complication. A sparse reward setting offers the cleanest isolation, but it cuts both ways. When reward only arrives at the end of a completed tier sequence and when max tier is set high enough the agent rarely sees one at all, leaving little signal to learn from. Dense or shaped rewards soften this problem but introduce their own noise and blur the picture of what the agent is actually learning from [3].

This points to a deeper asymmetry. A human dropped into a sequential collection task would figure it out quickly regardless of chain length, because we arrive with priors (those shared underlying rules already learned): an understanding of sequences, causality, and goals. An RL agent has none of that. It's learning the concept of "do things in order" from scratch, at the same time as learning which things and in what order, all from a single reward signal. One way to think about this is to expect a newborn to learn the same task from the same reward signal. This tension is itself a confounding variable that can be minimized by reducing sequential complexity, leveraging temporal distance as a cleaner axis of difficulty.

D. Markovian Decision Process and Partially Observable Markov Decision Process

A Markovian Decision Process (MDP) is a formalization of the RL problem where the current observation alone is sufficient to act optimally [6, sec. 3], not because the agent

is handed the answer, but because *once the policy is learned*, nothing from the past is needed to make the right decision. SYNGrid operates as an MDP by giving the agent a full view of the grid, where all orb positions and their identities are visible at every step.

A Partially Observable Markov Decision Process (POMDP) is the more general case where the observation is incomplete [6, sec. 17.3]. SYNGrid supports this too, through an agent-centric 3×3 window inside a larger grid acting as a fog of war. As the agent moves, the window moves with it, and anything outside is invisible.

Which observability condition the agent operates under determines whether spatial reconstruction becomes a memory demand on top of sequential reasoning.

E. Proximal Policy Optimization

Proximal Policy Optimization (PPO) is an on-policy algorithm, meaning it learns directly from experience generated by its own current policy [7]. At each update step, the agent collects a fixed batch of environment steps which is called a rollout, then updates its policy on it. To keep updates stable, PPO clips each update so the policy cannot stray too far from where it started. This is the proximal constraint the name refers to. Once the update is done, the experience is discarded and the process repeats with a clean batch of steps and the updated policy.

PPO also uses an actor-critic architecture [6]. The actor is the policy itself which selects actions. The critic runs alongside it, estimating the value of each state by predicting how much cumulative reward the agent can expect from that point onward. The difference between what the critic predicted and what actually happened — the advantage — is the primary driver for the policy update.

Early in training this signal is large and noisy because the critic is inaccurate, but that is not a problem because a large positive advantage tells the actor to do more of that thing, a large negative advantage tells it to do less. Over time the signal stabilizes as the critic improves, which is observable through metrics such as entropy loss and explained variance, which reflects how accurately the critic estimates returns. Stabilization signals that the agent has confidently learned something, whether that's good or bad only becomes clear when read alongside the reward curve.

1) Memory architectures: All three agents use PPO as their base algorithm. They differ only in their NN architecture or how the observation is constructed, which essentially changes how past experience is made available to the policy and not in the learning algorithm itself. This keeps memory capacity as the sole variable under comparison, and matters because memory becomes the determining factor only when the task pushes beyond what the algorithm alone can bridge.

The stateless PPO agent has no memory beyond the current observation. Each action is selected solely from what is visible at that timestep, whether in training or evaluation. During training each rollout nonetheless provides a window of experience over which the advantage is estimated, balancing immediate and future rewards through Generalized Advantage Estimation (GAE). This reflects how the agent learns sequential structure, not through memory, but through updates driven by repeated rollouts. Once the rollout is discarded that history is gone, and only the updated policy remains.

Frame Stacking Proximal Policy Optimization (FSPPO) extends the effective observation by concatenating the last n observations into a single input, giving the agent a short, fixed window into its recent past. Originally developed for pixel-based Atari environments [8], where a single frame is insufficient to infer trajectory of entities in the game. While this isn't the primary purpose for symbolic observations, it can still provide useful temporal context where the current state alone is ambiguous, for example by reconstructing the incomplete view of the grid in the agent-centric 3×3 observation mentioned earlier. Crucially though, frame stacking does not learn a compressed representation of history, it simply widens the input. The window is fixed, slides forward with each step, and older observations fall off entirely.

Recurrent PPO replaces the fixed window with a LSTM layer that maintains an episodic memory — a hidden state that accumulates context across timesteps within an episode and resets at each episode boundary [9]. The learned weights carry over between episodes, gradually encoding how to use that episodic memory effectively. Unlike frame stacking, the LSTM decides what to retain and what to discard, allowing it to carry relevant information across sequences of arbitrary length in principle. The tradeoff is computational: LSTM is slower to train and requires more wall-clock time to converge than both its stateless counterpart and the frame stacking version.

In summary, the three agents represent distinct memory scopes: the stateless agent acts on the current observation alone, the frame stacking agent extends this with a fixed window of recent observations, and the LSTM agent maintains a compressed, learned summary of the episode in its hidden state, deciding what to retain and what to discard as it goes. All three are established methods rather than state of the art and are chosen because they represent well-understood and distinct approaches to memory.

V. Related Work

Most benchmarks capable of probing TCA do not isolate it. Environments like VizDoom [4] entangle CA with exploration, inventory management, and combat, so if an agent fails, it is rarely clear which of these was the bottleneck [3]. Even among more controlled settings the problem persists. In Atari's *Seaquest*, rescuing divers introduces a delayed payoff because rewards for it are only given upon surfacing. This creates a

complex trade-off for the AI, which must balance immediate shark-shooting rewards against the long-term score multipliers unlocked by rescuing those divers, and this adds noise to the TCA signal it inherently contains. *Deferred Effects* in DeepMind Lab takes a similar approach: flipping a light switch imposes a penalty but lights up higher-reward cakes in a neighboring room for the agent to consume [3, sec 7.2.2]. The penalty on the switch adds reward shaping to the mix, now the agent must accept an immediate cost for a later gain, an interesting demand in its own right, but one that sits alongside the CA challenge rather than isolating it. Despite containing genuine CA challenges, these tasks were not designed to isolate them, and Pignatelli et al. reflect this gap plainly, calling for benchmarks that disentangle CAP from exploration and isolate it as a standalone challenge [3, sec. 8.2].

Two benchmarks sit closer to that goal and are most relevant to SYNGrid.

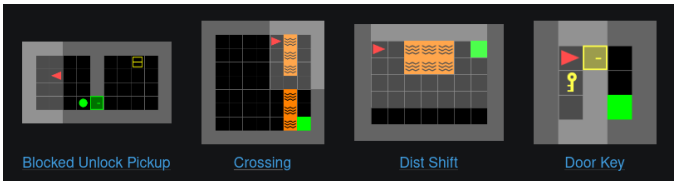

Fig. 1. Examples of environments from Minigrid[2]

MiniGrid [10] with its first commit to the repository[3] dating back to December 2017, continues to see active development and research use. Like SYNGrid, it's built on the Gymnasium framework and offers a variety of small grid environments[4]. Examples of these can be seen in Figure 1. Agents face tasks such as remembering the shape of an object in one room and selecting it later, using a key to unlock a door to collect an orb, or following an instruction to pick up the correct object among two options. Each environment isolates a specific aspect of exploration or delayed reward. The environments are lightweight and the tasks are interpretable, which makes MiniGrid a useful diagnostic tool.

Its limitation is that each task lives in its own environment. Testing a different mechanic means switching environments entirely or constructing your own from scratch. There is no shared layer that lets mechanics combine or compound. Scenario variety comes mainly from breadth of environments rather than depth of configuration within one.

Another highly relevant benchmark is **MiniHack** [11] (Examples of levels can be seen in Figure 2). It is built on top of the NetHack Learning Environment (NLE)[5], itself based on the classic NetHack game which is a turn-based, ASCII-graphics roguelike where players explore procedurally

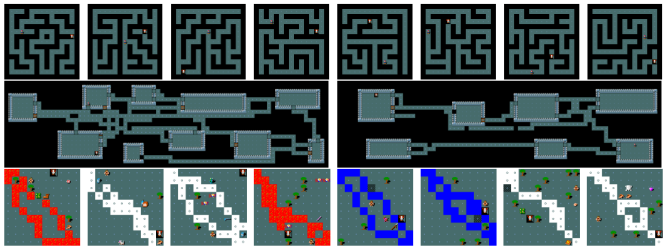

Fig. 2. Examples of MiniHack environments, taken from Samvelyan et al. [11]

generated dungeons, manage resources, fight monsters, and solve puzzles under permadeath conditions.

The core idea behind MiniHack is to leverage the rich mechanics of NetHack while providing researchers with highly configurable, MiniGrid-style environments [6] that can be easily adjusted and extended using `.des` files, a probabilistic description language. This allows researchers to create rich scenarios with sparse rewards, long action sequences, monsters, traps, and procedural generation, making it well suited to tasks with multi-step dependencies and distant feedback. However, tasks can draw on up to 75 context-sensitive actions spanning movement, inventory management, combat and object interaction, making it difficult to isolate CA from perception and navigation challenges. MiniHack does however support a modular action space, allowing researchers to restrict available actions per scenario.

The tradeoff is again complexity in designing custom tasks. While writing `.des` files is faster than incorporating your own environment through the code-base, it requires learning a dedicated language, and the underlying NetHack mechanics introduce perceptual and navigational demands that are difficult to separate from the CAP itself. The computational cost reflects this: IMPALA baselines typically require around 10M steps to solve the KeyRoom-Random-S5 [7] task, where the agent must locate a key, unlock a door, and reach a staircase. A challenge where three dependent sub-goals must be completed in sequence before any reward is given. A comparable three-tier orb chain in SYNGrid follows the same structure: collect tier 1, then tier 2, then tier 3, with reward withheld until the full sequence is complete. Standard PPO converges on this in roughly 800k steps. The action and observation spaces are not equivalent though, SYNGrid's action space consists only of the four cardinal directions, with consumption happening automatically when the agent moves onto an orb. And the agents differ as well, so the comparison is not direct. The gap, nonetheless, suggests that meaningful sequential dependencies can be constructed at substantially lower computational cost when the environment is simpler by design.

The gap both benchmarks leave is a single configurable environment where task complexity can be varied by combining

[2] https://minigrid.farama.org/
[3] https://github.com/Farama-Foundation/Minigrid
[4] https://minigrid.farama.org/environments/minigrid/
[5] https://minihack.readthedocs.io/en/latest/about/nethack.html
[6] https://minihack.readthedocs.io/en/latest/envs/index.html
[7] https://minihack.readthedocs.io/en/latest/envs/navigation/keyroom.html

and toggling mechanics rather than switching environments or learning a dedicated description language. That is the space SYNGrid is designed to occupy, where scenarios can be built by composing and configuring mechanics rather than rebuilding from scratch.

VI. Research Methodology

This section describes how the current state of SYNGrid was built and evaluated. It opens with describing the artifacts of study, the framework itself and the agents used to evaluate it, then moves on to the research approach, followed by an account of the iterative design cycles that shaped the final artifact. The experimental evaluation is then described in two parts: RQ1, which trains three agents with varying memory capacity across three scenarios to assess whether SYNGrid produces measurable and interpretable differentiation along different CA dimensions; and RQ2, which re-examines the same experiments through the lens of configuration intent, asking whether the observed outcomes are consistent with what each configuration change was designed to produce.

A. Artifacts

Fig. 3. SYNGrid single configurable grid where scenarios are created by adjusting world parameters and orb configurations rather than switching environments. The HUD displays score and step information for human playtesting and is not part of the agent observation.

1) SYNGrid: SYNGrid is a framework built on the Gymnasium interface[8], designed for benchmarking AI agents in various tasks, where its capability for isolating TCA is the target of evaluation for this thesis. The code and a reproduction package including all experimental configurations can be found on GitHub[9].

The core environment consists of a grid in which the agent navigates and consume orbs to gain rewards as shown in Figure 3. To isolate the CAP, only tier orbs are used in this thesis. SYNGrid does however support a modular orb system where additional orb types can be introduced. Currently two types are implemented: direct orbs, which yield rewards upon consumption, and tier orbs, which must be consumed in a specific order to yield rewards.

The framework supports modular observation spaces to accommodate agents of varying capacity. At the time of this thesis, two perception types are available: a 1D vector via Gym's `Box` space and a composite observation via `Dict`. The vector and composite observations each come in a MDP and a POMDP version. The MDP observation contains a complete top down view with agent and orb positions as well as orb identities, the POMDP version creates a draw distance by only giving the agent a 3×3 agent-centric window in the bigger grid.

The framework also contains an abstract `BaseAgentRunner` class that provides shared infrastructure for training and evaluating agents, such as managing output directories and logging. Next to this the `HumanRunner` lives which can be used to tune and get the feel of certain tasks that are being created.

At the heart of SYNGrid lies a YAML Ain't Markup Language (YAML) configuration file through which the environment can be tailored to create unique experimental scenarios. Grid dimensions, tier complexity, reward structures, orb types and agent observations can all be controlled from a single file, including saving scenarios for later use, making it straightforward to reproduce exact experimental conditions or explore new ones without touching the underlying code.

Lastly, a standalone plot module is also part of the framework, enabling drawing plots of both training and evaluation runs which loads Comma Separated Values (CSV) files generated by a custom `RecordEpisodeStatistics` for extracting data points specific to the custom environment.

2) Agents: The three agents with varying memory capacity introduced in the background are employed to empirically validate SYNGrid, using implementations from Stable-Baselines3 (SB3) and its extension SB3-Contrib. The stateless and frame stacking agents use `PPO` from SB3[10], while the recurrent agent uses Recurrent Proximal Policy Optimization (RPPO) from SB3-Contrib[11].

B. Research Approach

This study is methodologically grounded in a Design Science Research (DSR) approach, which is appropriate because the primary contribution is the SYNGrid artifact, whose value in isolating TCA ultimately is assessed through empirical evaluation. However, DSR is used here not as a contribution claim but rather as a process discipline, a way of ensuring that

[8] https://gymnasium.farama.org/
[9] https://github.com/J-manLans/SYNGrid
[10] https://github.com/DLR-RM/stable-baselines3
[11] https://github.com/Stable-Baselines-Team/stable-baselines3-contrib

the SYNGrid artifact is empirically grounded and not a result of relying on intuition or one off design choices.

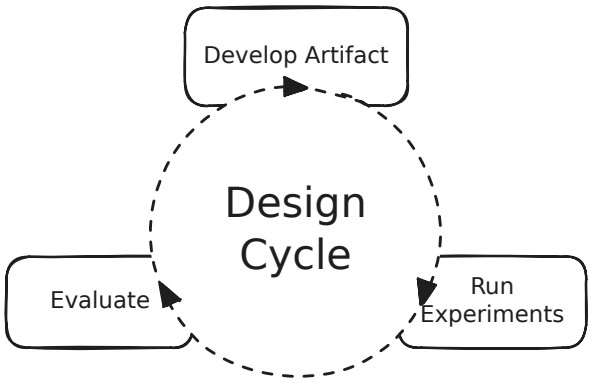

Fig. 4. SYNGrid's iterative design cycle begins by introducing a single change, then running experiments to see whether the intended agent behavior emerges. When it does not, the codebase is examined alongside further experiments and prototyping to identify the root cause. The findings then inform the next cycle

The internal DSR cycle follows Hevner and Chatterjee's build-evaluate paradigm [12], instantiated here as an iterative loop of development, experimentation, and analysis, and Figure 4 visualizes this flow. The cycle was used as follows: each candidate design was implemented and evaluated by training the PPO agents. When training or evaluation revealed poor performance such as failure to learn or persistently suboptimal behavior, that finding forced the diagnostic question: *Why isn't this working?* Answering that question required a diagnostic phase to determine the root cause. This involved exploratory prototyping, followed by designing targeted follow-up experiments aimed at isolating variables. Ultimately, these steps led to a new design approach. Therefore, the subsequent design was not arbitrary but driven by empirical observations from the previous cycle. In the end, this iterative process led to a suite of scenarios capable of producing measurable differentiation across TCA challenges, succeeding in partially isolating it from exploration and perception.

Following these iterative cycles, the resulting SYNGrid's utility is assessed through two lenses. Quantitative simulation tests whether it produces measurable differentiation across agent architectures under controlled conditions. Structured demonstration shows that meaningful task variation is achievable through configuration alone. Since all experiments are conducted in a computational setting, results are fully replicable from the provided configuration files.

C. Iterative Design and Artifact Development

The core SYNGrid environment was implemented in a prior course. This thesis extended it with three structural additions: a centralized YAML configuration file, a modular observation space and a base agent infrastructure. The configuration system replaced scattered hardcoded variables with a structured, Pydantic-backed model, making scenarios reproducible and modifiable without touching underlying code. The modular observation and agent interfaces were designed with a similar goal, different agent types can make use of the existing base, utilizing its helper methods to structure the experimental setup, and observations can be swapped to test different capabilities of the agent. For example, the POMDP variant tests spatial reconstruction from a partial observation, while the MDP version creates a cleaner TCA signal. Additionally, some agents require special types of observations, and the modularity is designed to cater to such requirements in future extensions.

These additions provided the foundation for the iterative cycles that shaped the final scenarios. Changes across observation design and reward structure often informed each other as insights from one area would suggest modifications in the other, and the process was not strictly linear. But before introducing further variables, each modification was stabilized and evaluated independently (including running the test suite and fixing any errors that emerged). This structure made it possible to attribute improvements to specific design decisions, avoiding the common problem of confounding multiple changes and losing traceability.

The observation space evolved significantly during this thesis. Early versions included, alongside agent and orb positions, features such as current score, tier chain progress, steps remaining in the episode, and orb identity expressed as separate category, type, and tier combination. The final observation is substantially slimmer with only agent position, orb positions, and a scalar-encoded orb identity computed from the category, type, and tier combination. The redundant features were removed through ablation, those that did not contribute measurably to convergence were dropped.

The biggest impact however, was moving from slot-based to a distance sorted orb order where orbs nearer to the agent appear earlier in the observation. A suspicion had been surfacing repeatedly that slot-based ordering forced the agent to learn fixed slot-to-orb correlations that carry no natural meaning, and paired with the prospect of a slimmer observation, the switch eventually seemed worth the effort. Distance sorting removes that burden, providing a simplified artificial analog of depth perception. A practical consequence that follows from this is that only active orbs need to be included in the observation and its size can therefore be limited to the maximum number of active orbs, rather than the total orb pool that was demanded for the slot-based setup. This reduces input size, removes irrelevant information, and enables curriculum training for setups where the max active orbs variable is constant.

The orb identity representation evolved through several iterations as well. Initially, identity was represented as a scalar combination of category, type, and tier. After the switch to distance-sorted observations, experiments with one-hot encoding were conducted, a representation where each possible orb identity gets a unique binary vector with a single 'hot' (1) bit and the rest zeros. The goal was to replace

the three scalar identity values with a discrete, non-ordinal representation — something one-hot provides naturally. And it proved to produce a cleaner and faster convergence, but it didn't scale, as the number of orb types increased (e.g., from tier 3 to tier 10), the observation space grew proportionally, and performance degraded.

This trade-off led to the final design: a scalar encoding that computes a unique identifier for each orb class from the combination of values described above. The scalar encoding preserves the cleaner signal of one-hot as the agents learn a single value rather than composing identity from multiple fields, while keeping the observation space fixed regardless of how many orb types are present. The trade-off is that scalar encoding returns a magnitude, not a pure identity, but the scalability benefit was judged more important for the benchmark's goals.

With the observation space covered, the remaining iterations concerned scenario balance and reward structure. Beyond agent metrics, human play-testing was used throughout to catch balance issues early, such as orb crowding caused by grid proportions that were too small, and scenarios that felt trivially solvable or effectively unlearnable.

Reward structure was tuned iteratively alongside this for each scenario. For example, a strictly sparse reward that's given only upon reaching max tier in a single chain mode where all orbs spawn in at the beginning, produced slow and unstable convergence. This was especially evident at higher tier configurations, where completed chains are rarely stumbled upon by chance. Consequently, three small penalties were introduced: movement, boundary, and wrong-order consumption. The boundary and wrong-order penalties were relatively clean additions, the former nudges the agent away from walls without meaningful side effects while the latter improved convergence speed by signaling incorrect consumption order, at the cost of slightly muddying the TCA signal. The movement penalty is more double-edged, it discourages looping but introduces a cost-of-living pressure that can interact poorly with other variables, as other scenarios later illustrate. However, evaluation runs with and without the movement penalty showed a small difference in mean episode length, and it was retained for the scenarios it didn't directly penalize — though in hindsight, the movement penalty was likely the wrong choice for single chain scenarios. As dependency length grows, looping increases naturally, and the intention was to flatten that curve. But the cost-of-living pressure it introduces creates its own confound, and a cleaner approach would have been to let looping be a signal of the agent struggling rather than attempting to penalize it away.

In another scenario, where consuming an orb triggers a delay during which no further consumption can occur, the movement penalty was discarded, the wrong-order penalty decided to be raised up to -1 and termination on wrong-order consumption was dropped. The rationale was that the increased temporal distance from action to reward, which grew from roughly 15–20 steps to a minimum of 30–60, depending on tier and delay, changed the nature of the information provided by intermediary penalties. With a movement penalty active, the agent never explored enough to discover a complete chain. The least costly behavior was walking straight to the nearest orb regardless of order, a wrong-order consumption followed by termination was cheaper than continued movement. When the wrong-order penalty was raised to counter this, the agent responded by looping in place, which now became the least costly behavior. To encourage exploration, the termination on wrong-order consumption was removed. Removing termination on wrong-order consumption broke that deadlock as the agent now could explore freely, and the raised penalty provided enough signal to make order matter.

D. RQ1

To answer RQ1, the three agents — stateless PPO, FSPPO, and RPPO — are trained across three scenarios, described in more detail below. Each scenario stresses a different aspect of TCA: spatial memory under partial observability (Scenario 1), reward sparsity versus reward density (Scenario 2), and temporal delay with empty visual feedback (Scenario 3), each corresponding to obstacles in solving the CAP identified by Pignatelli et al. [3, sec. 5.5]. Below, the overarching experimental setup will be described, followed by the staged procedure for each scenario which details how data will be collected and the rationale behind them. The extent to which these scenarios succeed in isolating TCA rather than merely correlating with it will be assessed from the results.

	Stateless	**Frame Stack**	**LSTM**
Memory	None	Fixed window	Learned hidden state
Algorithm	PPO	PPO	RecurrentPPO
Trained on	CPU	CPU	GPU
n_steps	128	128	128

TABLE I
AGENT CONFIGURATIONS ACROSS THE THREE MEMORY ARCHITECTURES

1) Experimental Setup: Memory architecture serves as the diagnostic variable for TCA and the specific configuration of each agent is summarized in Table I. The reasoning going in was if added memory improves learning, that indicates the task demands temporal reasoning the current observation can't supply. The stateless agent is not a null baseline against which to measure this. Even without memory, slow improvement is possible through pattern accumulation across rollouts, and a steep climb after first discovery, where few successful trajectories convert quickly into reliable behavior is itself a strong indicator of genuine CA, consistent with Pignatelli et al.'s argument that better CA methods have better ability to extract reliable signal from a sparse reward landscape [3, sec. 5.4]. Whether that climb belongs to the stateless or the memory agents is what the scenarios are built to reveal.

To evaluate if this is the case, the three agents were compared using implementations from SB3 and its contrib extension. SB3 was chosen for its out-of-the-box compatibility

with Gymnasium and straightforward interface. Since this thesis focuses on evaluating the benchmark rather than novel AI methods, accessibility was prioritized over framework flexibility. PPO was selected as the base algorithm because SB3-contrib provides a recurrent variant of the same algorithm, yielding three agents (stateless, frame-stacking, and LSTM) that differ only in memory capacity, keeping the comparison focused on memory architecture rather than algorithmic differences.

All three agents receive identical observations: a 1D vector providing agent position, all orb positions, and the scalar-encoded identity of each orb, either a MDP version with a complete top-down view, or a POMDP version with an agent-centric 3×3 window that moves with the agent. The 1D vector was chosen over the composite Dict observation for two reasons: computational efficiency, as vector observations require less processing overhead; and scope, as fully leveraging the Dict observation requires custom feature extractors that fall outside the time constraints of this work. Since all architectures share the same default feature extractor and receive identical observations, any representational limitation from this choice affects all agents equally.

Hyperparameters were kept consistent across agents so that memory capacity remains the primary variable. Fixed shared hyperparameters constitute a fair basis for comparison precisely because any limitation in the observation affects all agents equally. Where necessary, architecture-specific adjustments were made: frame count for the frame-stacking agent was set to 4 and LSTM hidden size to 256, both tuned on the spatial scenario and held constant across all three. Increasing frame count beyond 4 prolonged convergence without meaningful gain, and 512 hidden units added overhead without improving performance at the tier levels tested. `n_steps` and `batch_size` for the stateless and frame-stacking agents were set to match RPPO's defaults of 128. This exceeds the default maximum episode length of 100 steps, so each environment is guaranteed to complete at least one full episode per rollout. For delay scenarios where episodes may exceed this, `n_steps` is intentionally kept fixed as the rollout boundary becomes part of the experimental design, testing whether RPPO's hidden state can bridge temporal gaps that exceed what GAE alone can reach. `ent_coef` was raised to 0.025 across all agents to sustain exploration longer and `n_epochs` was set to 4 to cut training time, which over millions of steps became substantial at the default of 10.

Training utilized SB3's `DummyVecEnv` with 16 parallel environments to speed up the process. 16 was the point at which additional environments started to yield diminishing returns for RPPO, the most resource-intensive of the three, and therefore the natural upper bound. All experiments were run with three random seeds on a consumer-grade laptop. Stateless and frame-stacking PPO ran on the CPU, RPPO due to its recurrent architecture ran on the GPU.

The following conditions were held constant for all models across all experiments:

- Observation: 1D vector via Gymnasium's `Box` space (the observation is partial or full, POMDP or MDP depending on scenario)
- Hyperparameters: shared across agents (n_steps = 128, batch_size = 128, ent_coef = 0.025, n_epochs = 4), with architecture-specific adjustments where necessary
- Parallel environments: 16 during training
- Random seeds: 3, 7 and 42 ran on all scenarios
- Hardware: CPU for stateless and frame-stacking PPO, GPU for RPPO. All experiments were run on a consumer-grade laptop

2) Experimental Procedure: The three agents are each trained on the three scenarios introduced at the start of the RQ1 section and will be described in more detail below. All three scenarios operate in single-chain mode where all orbs spawn at random positions at the start of the episode and remain in the grid until consumed. The final reward is set to 10 in every scenario except the dense tier scaling variant, which uses a threshold scoring mode that pools the reward for each correctly consumed orb and pays it out when the chain breaks or completes. Consuming an orb out of order terminates the episode immediately except for the delay scenario. Training progress is monitored via TensorBoard, tracking reward, episode length, entropy loss, and explained variance. Training was conducted until convergence or plateau was clearly observed, constrained by the compute available on a consumer-grade laptop. A flat reward curve over 5 million steps served as the primary stopping criterion. However, this was treated as a soft ceiling — if supporting signals such as high entropy, low explained variance, or increasing episode length suggested the agent was still actively searching, training was extended beyond this point. Analysis is based primarily on the training curves, with post-training behavioral observation used to confirm identified failure points. Table II summarizes each scenario's configuration, and the complete YAML configuration files for each are provided in the reproduction package.

	Spatial	**Tier Scaling**	**Temporal Delay**
Observation	POMDP	MDP	MDP
Rewards	Reaching max tier	For reaching max tier only (sparse) or per correctly consumed orb (dense)	Reaching max tier
Penalties	Boundary, movement, wrong-order	Boundary, movement (sparse), wrong-order	Boundary
Episode Length	100	100	Delay dependent
Primary Axis	Grid size	Max tier	Delay

TABLE II
CONFIGURATION OF THE THREE EXPERIMENTAL SCENARIOS, EACH ISOLATING A DIFFERENT AXIS OF TCA DIFFICULTY: SPATIAL MEMORY UNDER PARTIAL OBSERVABILITY, TIER DEPTH, AND TEMPORAL DELAY

a) Spatial Scenario: This scenario stresses spatial memory under partial observability since the agent must integrate

observations over time to navigate a grid it cannot fully see.

Configuration:

- Observation: POMDP (a 3×3 agent-centric window)
- Rewards: Reaching max tier
- Penalties: Boundary, movement, wrong-order
- Episode length: 100 steps
- Primary axis: Grid size ($5 \times 5 \rightarrow 6 \times 6 \rightarrow 7 \times 7$)

Tier 3 was established through preliminary runs as the configuration where LSTM performed optimally while both stateless PPO and FSPPO started showing degrading performance, establishing it as the base for curriculum training. Grid size was then increased in two stages, from 5×5 to 6×6 and finally 7×7, progressively deepening the spatial challenge. Boundary, movement, and wrong-order penalties were retained to discourage looping and wall hugging, and to signal incorrect consumption order. The wrong-order penalty terminates the episode, meaning the full sequence must still be solved within a single episode and credit still has to travel the full length of the chain, even if the sparse setting is softened. The expectation is that LSTM will initially dip in performance as the grid expands but gradually recover, while stateless PPO and FSPPO continue to degrade. If this pattern holds, it demonstrates that SYNGrid can stress and distinguish memory capacity along the spatial axis as well as the sequential one.

b) Tier Scaling Scenario: This scenario pushes all three agents toward their performance ceiling from tier 4 and onward, increasing tier depth until learning breaks down. Stateless PPO was used as a probe to establish an upper bound, with memory architectures evaluated from one tier below that ceiling. They first operate in a sparse reward setting. This setting offers the cleanest TCA isolation, as reward arrives only upon completing the full chain. Once that ceiling is reached, reward density is increased to confirm that the dependency chain itself remains learnable — ruling out task impossibility as the cause of failure and isolating reward sparsity as the limiting factor. A human trial is also performed to establish a practical reward ceiling. The graphical scoreboard starts at 50, functioning as a life meter in other scenarios, meaning the final score minus 50 gives the approximate maximum reward an agent can expect under optimal play. This provides a concrete reference point for interpreting agent performance once the dense reward is introduced.

Configuration:

- Observation: MDP (full top-down view)
- Rewards: For reaching max tier only (sparse) or for each correctly consumed orb at chain break or completion (dense)
- Sparse penalties: Boundary, movement, wrong-order
- Dense penalties: Boundary, wrong-order
- Episode length: 100 steps
- Primary axis: Max tier

The expectation is straightforward, as the dependency chain grows longer and reward becomes harder to stumble upon, performance should degrade. Introducing dense rewards at that ceiling is then expected to bring performance back, not only because the task got easier, but because the agent now gets told when it is on the right track. If that recovery holds, it wasn't the chain length that broke learning, but the silence. To support this comparison, the same penalty structure from the Spatial scenario was carried over here, with minor tuning. The most notable adjustment was removing the movement penalty in the dense reward condition since with it present, the agent showed lower initial exploration, and the concern was that cost-of-living pressure would cause it to settle for partial chain completion rather than pushing toward the full sequence, as discussed in Iterative Design and Artifact Development.

c) Delay Scenario: This scenario is the most direct test of LTCA in this thesis. After each correct orb consumption, all orbs disappear from the grid and only reappear once a forced delay expires. The agent must learn to consume the correct tier orb, wait through the silence, then chase tiers again. Reward is withheld until the full chain is complete. Consuming an orb out of order does not terminate the episode, instead a -1 penalty is applied, the same if episode steps runs out. Stateless PPO is used as a probe again, starting at a 30 steps delay and increasing until performance breaks down. That breaking point then becomes the delay for starting evaluation of the memory architectures. Episode length scales up to accommodate the delay and tier combination.

Configuration:

- Observation: MDP (full top-down view)
- Rewards: Reaching max tier
- Penalties: Boundary, wrong-order
- Episode length: Tier and delay dependent
- Primary axis: A fixed delay between possible orb consumptions

By training stateless PPO on the default delay in increasing tiers, tier 3 turned out to be the highest it could handle under said scenario without flatlining. With that established, the focus shifts to what happens as delay increases from there. Since the MDP observation allows stateless to infer sequence from orb identity alone, all agents were expected to handle moderate delay. The hypothesis was that as delay widens beyond what GAE can bridge, stateless would degrade toward suboptimal performance while LSTM, would maintain optimality because it is carrying hidden state across the full episode. Frame stacking was expected to offer no meaningful advantage here since its fixed memory window of 4 frames effective spans only a handful of steps, far short of the minimal delay lengths tested. Increasing the frame count to match the delay would expand the input space significantly, slowing convergence past the compute budget without providing the kind of compressed temporal representation that LSTM maintains naturally. In this scenario it should be effectively stateless, but much slower. Tuning this scenario required removing the movement penalty,

raising the wrong-order penalty to -1, and dropping episode termination on chain breaks, as discussed in the end of Iterative Design and Artifact Development.

E. RQ2

To assess RQ2, the behavioral outcomes from the RQ1 experiments are re-analyzed through the lens of configuration intent, evaluating whether each configuration change for the different scenarios produced learning outcomes consistent with their intended effects. Consistency is assessed on two levels: whether performance moves in the intended direction, and whether observed agent behavior can be traced back to the specific mechanic the configuration change introduced.

Three configuration comparisons from the RQ1 scenarios serve as the primary evidence:

- **Observability**: MDP versus POMDP observation at the same tier, intended to create a genuine memory demand by restricting what the agent can see at any given step
- **Reward density**: sparse versus dense reward at the same tier depth, intended to reduce difficulty by giving agents more frequent feedback
- **Delay length**: increasing delay at fixed tier depth, intended to isolate temporal distance as a standalone difficulty axis, making the same task progressively harder without changing what the agent needs to learn, only how far apart the relevant events are

For each case the most informative agent comparison is selected, focusing on the agents whose behavior best illustrates the intended effect of the manipulation, and the corresponding training curves from RQ1 are assessed for directional consistency. Agent behavior is also examined by running the fully trained checkpoint to determine whether it can be traced back to the specific mechanic the configuration change introduced. If outcomes align with intent across all three cases, this constitutes evidence that SYNGrid's configuration system produces reliable and interpretable behavioral variation and supports its use as a diagnostic benchmark tool.

F. Limitations and Threats to Validity

Several limitations apply to this study. First a few minor ones: Hyperparameters were kept fixed across agents for comparability rather than tuned individually, meaning stateless PPO and FSPPO may not have been evaluated at their full potential. Experiments were run with three random seeds, providing limited statistical confidence, and compute constraints imposed a soft ceiling on training duration. However, since the goal is to evaluate the benchmark's ability to produce measurable differentiation between architectures rather than to find optimal agent performance, these constraints affect the precision of the findings rather than their validity.

Second: all three agents operate within the PPO algorithm family. While this made comparisons easier, the findings about memory architecture are therefore specific to rollout-based policy gradient methods and may not generalize to other approaches.

Third: the scalar encoding of orb identity may leak ordinal information to the stateless agent. Because the encoding increases with tier, its magnitude tracks the required consumption order directly, potentially letting the agent exploit that gradient rather than learning the sequence through genuine CA. While this affects all agents equally in terms of observation representation, it may disproportionately benefit the stateless agent at higher tiers where the ordinal gradient becomes a more meaningful crutch.

Fourth: the assessment of RQ2 relies partially on qualitative behavioral observation rather than formal measurement. While these observations are consistent with the intended mechanics, they cannot be quantified with the same confidence as training curves, and should be interpreted as supporting evidence rather than conclusive proof.

Fifth: the movement penalty in the three scenarios introduces a cost-of-living pressure that may interact with the TCA signal, the extent of which was not fully isolated within this work.

Sixth: a further limitation concerns replicability. During final repository preparation, the recurrent agent's performance on the spatial scenario could not be reproduced despite identical seeds, while the other two agents and all other scenarios were unaffected. That the failure is specific to the recurrent architecture suggest a problem at episode boundary handling have been introduced sometime after the initial runs, though the cause wasn't isolated before the deadline and can't be taken for granted. The original logs and checkpoints are preserved and remain the basis for the reported results.

Finally: the scenarios were designed primarily through an iterative design science process, with less explicit grounding in theoretical frameworks than would have been ideal. Pignatelli et al. identify the three MDP dimensions depth, density, and breadth, whose pathological conditions give rise to three corresponding CA challenges: delayed effects, sparse rewards or low action influence, and credit dilution [3, sec. 5]. Of these three challenges, delayed effects is the most extensively covered, addressed both through the tier chain mechanic and the dedicated delay scenario. Low action influence through low goal density is directly addressed in the tier scaling scenario, where the goal is only reachable through a specific sequence of actions that becomes exponentially rarer to discover as tier depth grows. Credit dilution however is absent from the experimental design. This was a deliberate choice given three scenarios were already in scope. Addressing it would require a continuous spawn mode, where intermediate tier orbs between tier 1 and max tier spawn and despawn throughout the episode, forcing the agent to disentangle which actions in a crowded landscape actually contributed to the final reward. The framework would support this mode with small modifications, but

exploring it fell outside the scope of this thesis.

VII. Results

A. RQ1

Three agents — stateless PPO, FSPPO, and RPPO — were trained across three scenarios, each stressing a different dimension of credit assignment CA: spatial memory under partial observability, reward sparsity versus density at increasing tier depth, and temporal delay between orb consumptions.

The movement penalty was double-edged as it discouraged looping but introduced a cost-of-living pressure that may itself have shaped behavior, as discussed in Iterative Design and Artifact Development. Residual looping persists, especially as tier depth grows, which is the main reason reported rewards sit slightly below the theoretical optimum throughout.

All experiments were run across three seeds (3, 7, and 42). Since seed variance showed no meaningful difference in behavior, results are reported for seed 3 for clarity. Full results across all seeds are available in the reproduction package in the GitHub repository[12]. For clarity, training curves are visualized using TensorBoard smoothed values, and reported performance values are approximated from these curves for readability. Results are reported per scenario below.

1) Spatial Scenario: For this scenario, training was stopped if mean reward showed no meaningful improvement within 1M steps. A preliminary run extending beyond 18M steps for stateless PPO confirmed that while performance continued climbing slowly, it never approached RPPO ceiling and recovery after each grid size increase was even slower. Since the pattern persisted across extended training, the compute budget was kept as a stopping criterion and the results below reflect this.

Figure 5 shows RPPO making a slow start before climbing from approximately 1M steps, converging near the optimal reward of 10 (≈ 9.1) around 7M steps for the initial 5×5 configuration. Stateless PPO and FSPPO converge at suboptimal rewards around 5.2 and 6 respectively. As grid size increases, both stateless PPO and FSPPO continue to degrade, while RPPO dips before recovering close to optimal performance. Across increasing grid sizes, RPPO also shows slightly higher explained variance compared to the other agents which declines even further for each increase in size.

Figure 6 reveals opposite search strategies between the architectures. All three agents begin with similar episode lengths, but diverge rapidly. RPPO extends its episodes as it explores the expanded state space before settling into shorter, more directed paths as the policy converges. Stateless PPO and FSPPO drop quickly to short episodes from frequent chain breaks, then gradually recover length as they partially learn the sequence. As grid size increases, stateless PPO and FSPPO episode lengths continue to grow without the same settling behavior RPPO exhibits.

2) Tier Scaling Scenario: All three agents were evaluated in a sparse reward setting with increasing tier depth. Stateless PPO was used as a probe to establish an upper bound which was found at tier 7, with memory architectures evaluated from one tier below that ceiling to avoid redundant compute. While FSPPO flatlined on the same tier as the stateless model and with slightly lesser performance on tier 6 where it managed to get rewards (Figures 8 and 11), RPPO flatlined at tier 6 and had to be run at tier 5 where it managed to solve the dependency chain (Figures 9 and 12).

Notable is that the reward at convergence declines steadily with tier, evident from stateless PPO probing runs (Figure 7), while episode length initially grows until tier 7 where it collapses to a low mean of 3.8 (Figure 10), consistent with frequent wrong-order consumption and immediate chain breaks rather than extended exploration.

When dense rewards were introduced at each architecture's failing tier, all three recovered. Figures 13–18 shows the learning curves from the dense runs and Figures 19 and 20 combine them for direct comparison, showing stateless PPO as the most sample efficient. This is also evident in the length curve, which peaks early during its climbing phase before stabilizing around FSPPO's level. RPPO's climb is markedly less steep by comparison. However, compared to the human baseline, they all underperformed as seen in Table III.

Agent	Human reward	Agent reward
Stateless PPO (tier 7)	∼207	∼169
FSPPO (tier 7)	∼207	∼162
RPPO (tier 6)	∼141	∼109

TABLE III
Agent performance versus human baseline under dense rewards

Notably, examining the trained models directly revealed an additional behavioral looping pattern, where the agent moves back and forth between two cells, that increased steadily with tier depth during PPO's probing run: once at tier 4, twice at tier 5, and three times at tier 6 out of ten evaluation grids. When dense rewards were introduced at the failing tier, failure modes persisted: stateless PPO got caught looping four times at tier 7, staying true to form; FSPPO looped once and failed to follow tier order once; and RPPO failed to follow tier order twice at tier 6.

3) Delay Scenario: This scenario had stateless PPO probing delays until noticeable degradation in performance was observed. The probing went through delay 30, 70, 130, 260 and 390 (see Figures 21 and 22). Degradation started right after the 30 delay (which showed clean convergence close to optimal performance), but became clearly visible at 260 and 390 where hills-and-valley patterns appeared with reward differences of up to 2 units. The agent still showed learning signal at both of the higher delays due to a steep climb

[12] https://github.com/J-manLans/SYNGrid

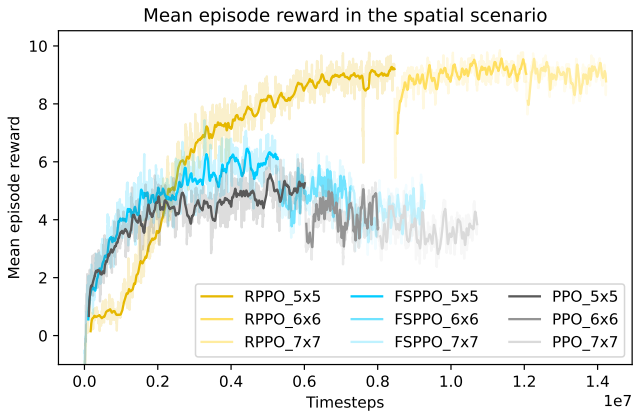

Fig. 5. Mean episode reward during training for stateless PPO, FSPPO and RPPO in the spatial scenario across increasing grid sizes

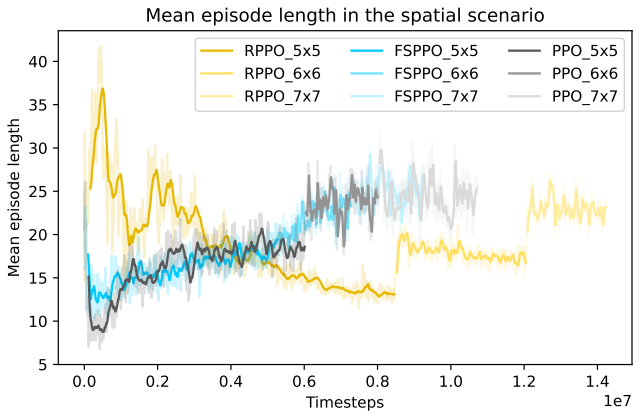

Fig. 6. Mean episode length during training for stateless PPO, FSPPO and RPPO in the spatial scenario across increasing grid sizes

relatively early in training and a high entropy, with explained variance staying around a low 0.2, suggesting the critic had not yet captured the structure of the task. The agent could likely have continued learning if pushed, but training was cut off due to compute and time budget, as the degradation was already clear enough to establish the breaking point.

On the same delay where training for stateless PPO stopped (390) FSPPO was trained, and it also started to learn in a similar hills-and-valley pattern but to a much lower rate, so when reaching approximately the same amount of steps as stateless PPO, training was aborted (see Figure 23).

Due to its slower wall-clock nature the recurrent agent was trained first at the lowest delay to confirm it would converge at all within any kind of compute budget before moving further. And after 10 hours it had, with a very similar reward curve compared with the stateless and frame stacking agent but delayed both in wall-clock time and sample efficiency, as shown in Figures 24 and 25. However, the difference in learning signal stood out. Compared to stateless PPO's and FSPPO's explained variance of ≈ 0.2, RPPO maintained a stable range between ≈ 0.87 and ≈ 0.96 after convergence, alongside noticeably lower and more stable entropy loss as well as Figures 26 and 27 show. This motivated running the next delay to see if RPPO's memory advantage would surface where stateless PPO converged to a suboptimal ≈ 7.5 compared to the ≈ 9.1 achieved at delay 30. RPPO did eventually converge, but the curves tell the same story as for the 30 delay, and at the next delay of 130, convergence proved to be beyond reach within the training budget, with flat reward and stochastic explained variance persisting across 33M steps.

When it comes to episode length, on delay 30, it stayed maxed out until reward began its steep climb, then dropped sharply as the chain was solved, with length curve mirroring the reward curve in reverse, which becomes apparent when comparing Figures 24 and 25. When looking at stateless PPO length curves in Figure 22, the same inversion holds across the increasing delays.

B. RQ2

The three configuration comparisons: observability, reward density, and delay, each produced outcomes consistent with their intended direction as the relevant curves and results from RQ1's scenarios show. Therefore, what follows only reports additional behavioral observations from evaluation runs.

Examining the trained models from the spatial scenario makes the memory advantage concrete. If RPPO passes tier 3 on its way to collecting tier 1, it will consistently take the shortest path back to tier 3 afterwards while stateless PPO and FSPPO keeps on searching for it as if they had never seen it. Figures 28 and 29 show this behavior for RPPO and FSPPO across different grids, while Figure 30 confirms RPPO's different behavior even on the same grid as FSPPO.

In the tier scaling scenario, evaluation runs of the trained agent revealed a consistent shift in failure mode with increasing memory capacity, namely less looping and more wrong-order failures, as noted in the end of the Tier Scaling Scenario results above.

In the delay scenario, RPPO's memory advantage manifests much as it did in the spatial scenario. It acts on information no longer visible, consistently moving toward where it last saw the orbs (Figure 31), while stateless PPO (Figure 32) and FSPPO loop in arbitrary positions as if that information were gone.

VIII. Discussion

The experiments revealed a more nuanced picture than the initial hypothesis suggested. This section interprets the experimental findings, discusses where SYNGrid goes from here, extracts lessons learned, and situates the results within the broader question of what tabula rasa RL can and cannot do.

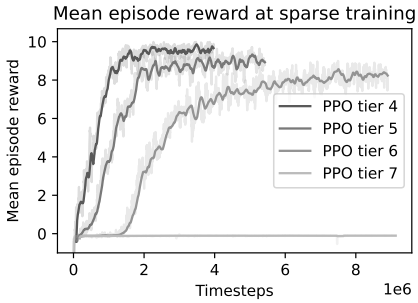

Fig. 7. Mean episode reward during training for stateless PPO in the tier scaling scenario under sparse reward conditions on tiers 4–7

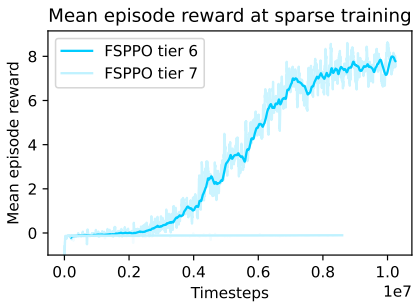

Fig. 8. Mean episode reward during training for FSPPO in the tier scaling scenario under sparse reward conditions on tiers 6–7

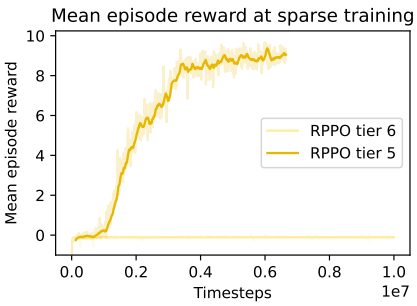

Fig. 9. Mean episode reward during training for RPPO in the tier scaling scenario under sparse reward conditions on tiers 5–6

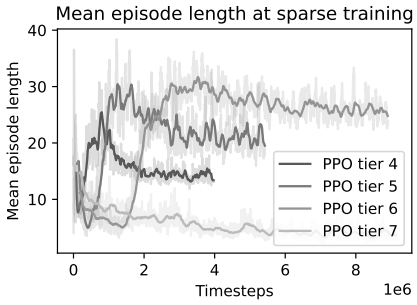

Fig. 10. Mean episode length during training for stateless PPO in the tier scaling scenario under sparse reward conditions on tiers 4–7

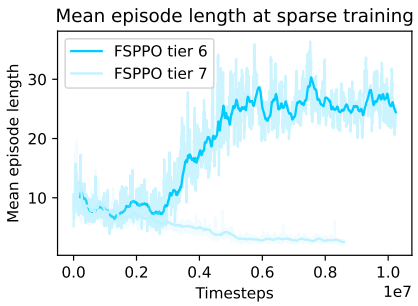

Fig. 11. Mean episode length during training for FSPPO in the tier scaling scenario under sparse reward conditions on tiers 6–7

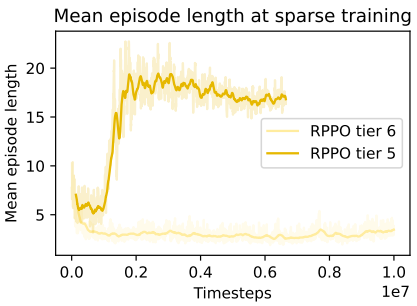

Fig. 12. Mean episode length during training for RPPO in the tier scaling scenario under sparse reward conditions on tiers 5–6

A. Benchmark evaluation

a) Summary of Findings: Memory mattered mainly where the observation withheld information, as in the spatial scenario's 3×3 agent-centric window. This is consistent with the expected distinction between fully and partially observable settings, and evidence that the benchmark reproduces it. In the fully observable scenarios, reward density proved the stronger determinant of learning: when dense rewards were introduced at each architecture's failing tier all three agents recovered, a concrete demonstration of the entanglement Pignatelli et al. describe, isolated as a single configurable variable. Pushing the delay between action and reward don't produce a flatline but a degrading curve of increasingly unstable hills-and-valleys, proof the signal still exists, but arrives too weakened to leave more than a ripple in performance. This shows that temporal distance degrades learning gradually rather than acting as a hard off-switch. Taken together, the three scenarios also demonstrated that meaningful CA challenges emerge from configuration alone. This lowers the bar for adoption, and SYNGrid's difficulty axes can locate breaking points or be tuned to match agents at different stages of development, making it a diagnostic tool rather than merely a scoreboard. In that sense, SYNGrid is one response to the kind of benchmarks Pignatelli et al. called for, and continued development, building on the findings of this thesis, will show whether it can genuinely aid in solving the CAP.

b) RQ1: The central question was to what extent SYNGrid isolates TCA with memory architecture as the diagnostic variable. The answer turned out to be: to some extent, but with the caveat that in fully observable settings reward density proved a stronger determinant of learnability than memory architecture, and recurrent memory contributed less than initially hypothesized.

When RPPO did converge however, explained variance was generally higher and entropy lower than its stateless counterpart, this is most clearly visible in the delay scenario as Figures 26 and 27 exemplify. This suggests that despite slow convergence, what emerged was a more coherent internal model of the task, and the behavioral traces support it as well: where stateless PPO loops in arbitrary positions through the delay (Figure 32), RPPO moves toward where it last saw the orbs (Figure 31). Whether that translates to an advantage at higher tiers or delays pushed further than current compute budget allows remains uncertain. In the sparse tier scaling scenario, RPPO flatlined on tier 6 despite running for 10M steps (Figure 9), and in the delay scenario RPPO only managed to handle the 30 and 70 delay while stateless and FSPPO found signal all the way up to at least delay 390 (see Figure 23). According to Pignatelli et al. common baselines (PPO being one) struggle to extract reliable signal from few successful trajectories [3, sec. 5.4], and the results suggest this applies equally to RPPO in a MDP with the added cost that the

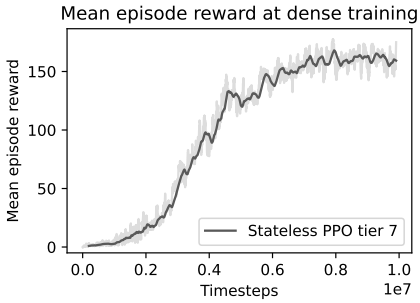

Fig. 13. Mean episode reward on tier 7 during training for stateless PPO in the tier scaling scenario when dense reward were introduced.

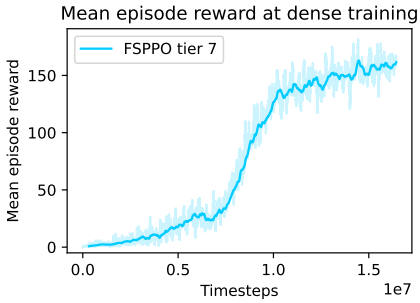

Fig. 14. Mean episode reward on tier 7 during training for FSPPO in the tier scaling scenario when dense reward were introduced

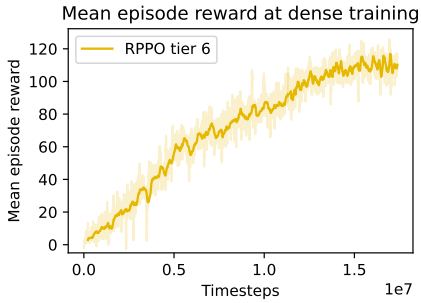

Fig. 15. Mean episode reward on tier 6 during training for RPPO in the tier scaling scenario when dense reward were introduced

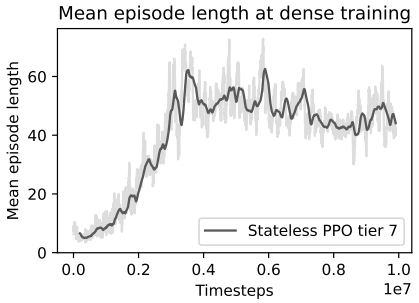

Fig. 16. Mean episode length on tier 7 during training for stateless PPO in the tier scaling scenario when dense reward were introduced

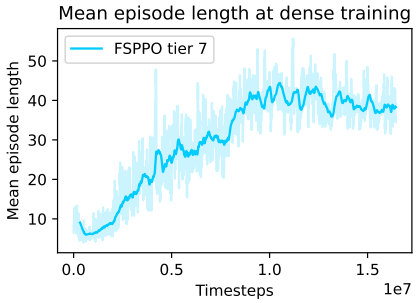

Fig. 17. Mean episode length on tier 7 during training for FSPPO in the tier scaling scenario when dense reward were introduced

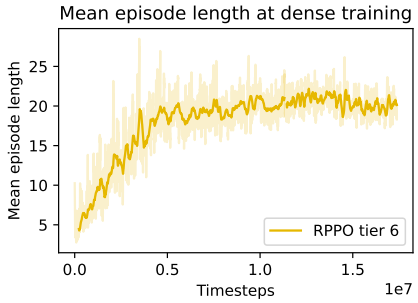

Fig. 18. Mean episode length on tier 6 during training for RPPO in the tier scaling scenario when dense reward were introduced

hidden state must also be shaped by that sparse signal before it becomes useful, stretching the silence further. Each scenario contributed a different piece of that picture.

The spatial scenario complicated clean TCA isolation by introducing a spatial inference demand alongside the sequential one while producing the clearest architectural differentiation. Fog of war creates a genuine memory demand that no amount of GAE sophistication compensates for, and as grid size grew this became increasingly visible: episode length curves grew noisier for the stateless and frame-stacking agents with each expansion as Figure 6 shows, consistent with search becoming less directed, supported by the path traces comparisons in Figures 28–30.

The tier scaling scenario confirmed that sparsity rather than chain length was the bottleneck. As tier depth increases, the proportion of trajectories in which the agent reaches the reward drops, thinning the CA signal. Pignatelli et al. identify the minimum rate of successful trajectories required for convergence as an open question in CA research [3, sec. 5.4]. The dense reward result illustrates the phenomenon they describe: learning recovered not because the chain changed, but because the rate of informative trajectories did. SYNGrid's configurable tier depth offers one way to narrow that bound empirically. In this thesis for example, stateless PPO converged at tier 6 but not at tier 7, placing the practical convergence boundary for this setup between a 1/720 and 1/5040 chance of random discovery ($1/6!$ and $1/7!$).

The delay scenario showed that GAE was more robust than expected. Sixteen parallel environments diversifying the batch across delay phases likely let GAE bridge temporal gaps that were expected to favor LSTM, even with rollouts spanning beyond n_steps. That said, RPPO showed higher explained variance at delay 30 (Figure 26) and 70, yet reward curves remained comparable. This suggests the architectural difference is real, but insufficient to compensate for a reward signal already in decay. Hung et al. define the half-life (strictly, the $1/\mathrm{e}$-life) of a discounted reward signal as $\tau = \frac{1}{1-\gamma}$, beyond which signal strength decays past roughly 37% and reliable updates become difficult [2]. Here γ, a value between 0 and 1, is the factor. A higher γ means distant rewards decay more slowly, and SB3 defaults to 0.99. At that value the half-life is 100 steps. Approximating the silence the gradient must cross as $d \times (t-1)$ where d is delay and t is max tier, only delay 30 stays under that horizon at roughly 60 steps. Table IV confirms this, consistent with the growing instability in the reward curves beyond it as seen in Figure 21.

What isolating TCA means in practice is less obvious than it appears. The scenarios touched two of Pignatelli et al.'s three CA dimensions [3], yet each produced confounding factors alongside the intended signal. The diagnostic value noted in the summary above remains nonetheless: locating where agents break down remains informative even when the

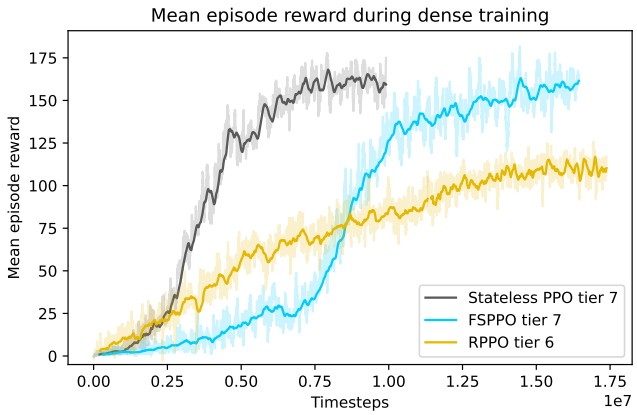

Fig. 19. Mean episode reward during training for all three agents combined in the tier scaling scenario when dense reward were introduced. Stateless PPO and FSPPO on tier 7, and RPPO on tier 6

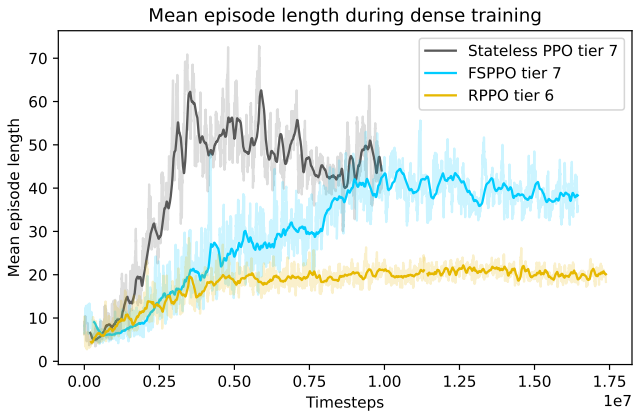

Fig. 20. Mean episode length during training for all three agents combined in the tier scaling scenario when dense reward were introduced. Stateless PPO and FSPPO on tier 7, and RPPO on tier 6

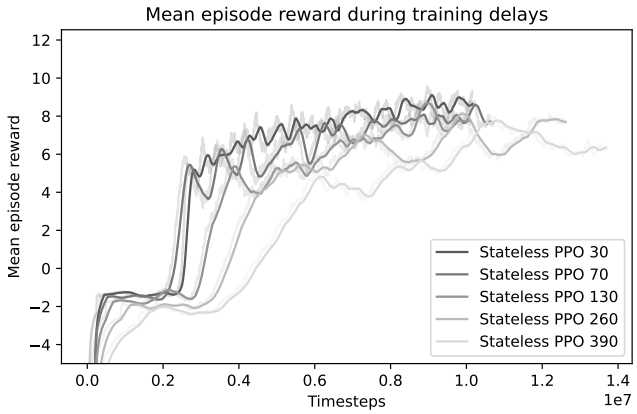

Fig. 21. Mean episode reward during training for stateless PPO across delay configurations 30, 70, 130, 260 and 390 on tier 3

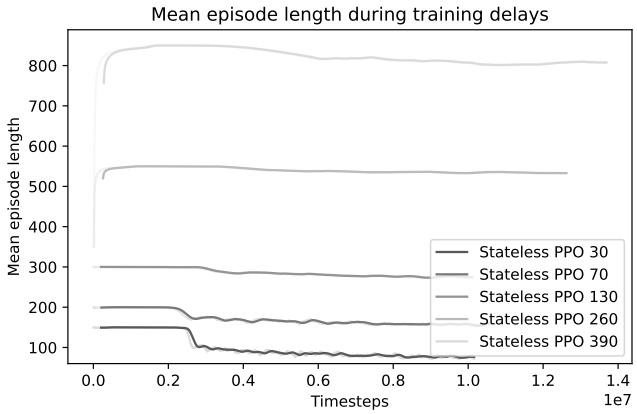

Fig. 22. Mean episode length during training for stateless PPO across delay configurations 30, 70, 130, 260 and 390 on tier 3

Delay	$d \times (t-1)$	Remaining Signal
30	60	$\gamma^{60} \approx 55\%$
70	140	$\gamma^{140} \approx 25\%$
130	260	$\gamma^{260} \approx 7\%$
260	520	$\gamma^{520} \approx 0.5\%$
390	780	$\gamma^{780} \approx 0.04\%$

TABLE IV
DISCOUNTED SIGNAL STRENGTH AT EACH DELAY ON TIER 3, $\gamma = 0.99$

breakdown has more than one cause.

c) RQ2: The training curves and results from RQ1 confirm that SYNGrid's configuration-driven design produced outcomes consistent with its intentions, and the agent behaviors that emerged make that concrete. Shrinking the observation to a 3×3 agent-centric window introduced a visibility limit that forced spatial memory to matter. Agents without it fumbled in the dark while RPPO mapped the area methodically, remembering where tier 3 was after collecting tier 2 and taking the shortest path back despite the darkness (Figures 28–30).

When rewards were stretched thin in the tier scaling scenario, introducing step-wise feedback per correct consumption gave agents enough signal to keep going. It also brought something else into view — without memory, each added tier pushed the stateless agent further toward looping in place, defaulting to apathy when no clear path forward existed. RPPO ventured further, failing the dependency chain rather than abandoning it as noted in the end of the Tier Scaling Scenario.

The increasing delay produced the intended gradual degradation rather than outright failure. As delay grew, the credit signal thinned with distance, and what the agent learned one rollout it struggled to hold onto the next. Where performance remained persistent, memory allowed RPPO to remember orb placements, effectively homing in on their last seen position, reducing episode lengths ever so slightly, as seen in Figures 31,32 and 25.

These three scenarios demonstrate that meaningful CA challenges can be produced through configuration alone, but

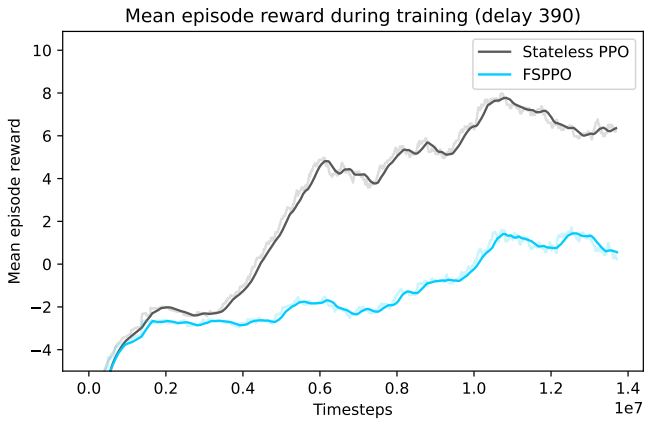

Fig. 23. Mean episode reward during training for stateless PPO and FSPPO in the delay scenario at delay 390 on tier 3

the deeper claim is about the design philosophy rather than the specific outcomes. Even in prototype form, with a single orb type, the config-driven design proved expressive enough to stress different CA dimensions in distinct and interpretable ways. The three scenarios in this thesis demonstrates the viability and suggests SYNGrid is worth developing further.

B. Future Directions

The most natural extension of SYNGrid is the implementation of the effect orbs, which is the mechanic the benchmark arguably was built toward. Where tier orbs demand sequential consumption, effect orbs create transient world states where the value of an action only becomes clear later through interacting with something else. Synergy orbs that nullify the next orb, converter orbs that flip rewards, hazard orbs that make regions costly to traverse or raise obstacles. These orbs can be combined and reconfigured without touching the base environment, which is where SYNGrid's combinatorial design becomes genuinely interesting.

To make SYNGrid accessible beyond this thesis, a dedicated Graphical User Interface (GUI) would reduce the cognitive overhead that the config file setup has proven to add. The Pydantic model already in place makes this feasible and a cleaner interface that shows only relevant variables for the current setup paired with a scenario loader and builder could give users a straightforward way to save configurations they create. With its help a curated lineup of diverse, isolated scenarios could more easily be created and bundled with the framework, giving users representative tasks to evaluate agents against out of the box.

C. Lessons Learned

Building and evaluating SYNGrid across repeated design cycles surfaced five lessons worth passing on to anyone attempting something similar. They are collected here, ordered from the most general to the most specific to this benchmark's domain.

a) Preserve the state that produced a result: The replicability issue described in Limitations and Threats to Validity surfaced after submission, but was most likely introduced during the final stretch when results needed recording and time for careful integration had run out. Late changes were stitched into an otherwise modular system rather than carefully implemented (each isolated, tested, and confirmed before moving on), yet the sum of them shifted the system in ways single checks couldn't catch. One change at a time helps in keeping the system steady for the task at hand, but in a bigger system the interconnectedness becomes exactly where even verified changes can drift. This shows the necessity of binding the exact code and configuration state to each accepted result, so the system can later be compared against a known good version. Growing the test suite alongside the framework to catch drift instead of accumulating it is likewise a practice that pays for itself.

b) Movement penalty confound: A step penalty was used with the intention to discourage the agent from looping in place. This penalty was carried over from an earlier survival-based scenario where it was load-bearing, but once the goal shifted from survival to solving the tier chain, it stopped being a mechanic and became reward shaping. Something that needed to be balanced against wrong-order penalty and dampened exploration, while only slightly offsetting the looping. This showed that penalties tied to discouraging discrete acts earn their place while a per step penalty in a scenario with no survival stake mainly just masked the looping, which was itself a signal of the agent struggling — and shaping it away removed that signal without addressing the cause. The pattern that emerges is one of entanglement. The boundary penalty is self-contained, it says "don't walk into walls" and means only that. The chain-break penalty is bound up with the task itself since it hints that order matters, which slightly muddies the very signal being measured, yet stays workable. The movement penalty is neither discrete nor self-contained, but instead fires every step, compounds with the other penalties, and once survival left the task it carried no load-bearing meaning at all, and this distinction is worth holding onto when deciding how to model a world for an agent.

c) Diagnostic utility over clean isolation: In attempting to build the isolated scenarios Pignatelli et al. call for [3], the confounds discussed above crept into every scenario, but they don't necessarily need to be a failure of isolation. Since they live in configuration rather than in fixed structure, isolation becomes approachable by subtraction: the same knob that potentially lets a wider range of agents learn at all also lets them be pushed to where they clearly cannot. This applies to any benchmark that isolates through predefined scenarios. Surfacing those axes explicitly, as settable parameters in the framework rather than something to dig for in the source code, makes difficulty something you can adjust freely. Though

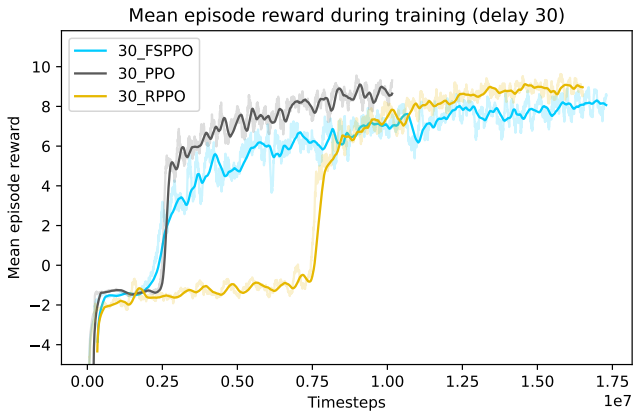

Fig. 24. Mean episode reward during training for stateless PPO, FSPPO and RPPO in the delay scenario at delay 30 on tier 3

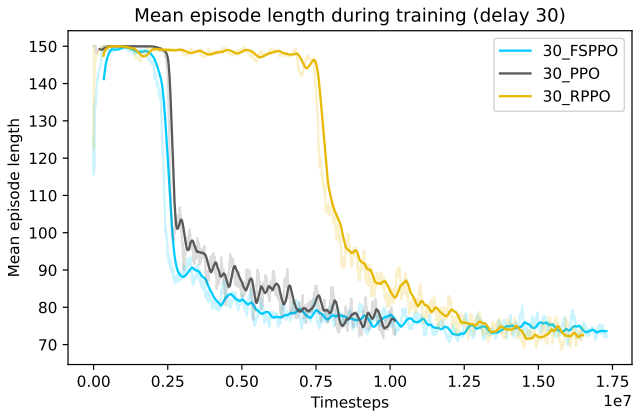

Fig. 25. Mean episode length during training for stateless PPO, FSPPO and RPPO in the delay scenario at delay 30 on tier 3

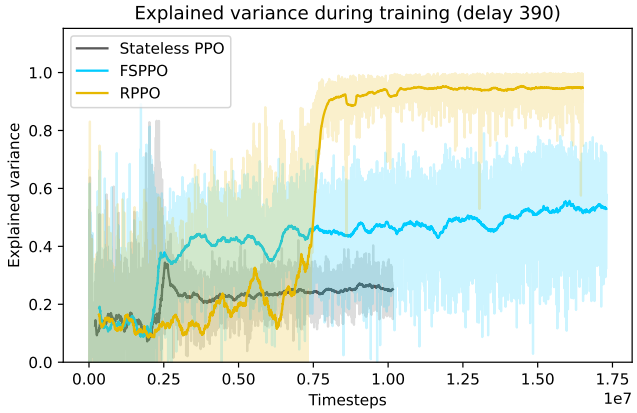

Fig. 26. Explained variance of the critic during during training for stateless PPO, FSPPO and RPPO in the delay scenario at delay 30 on tier 3

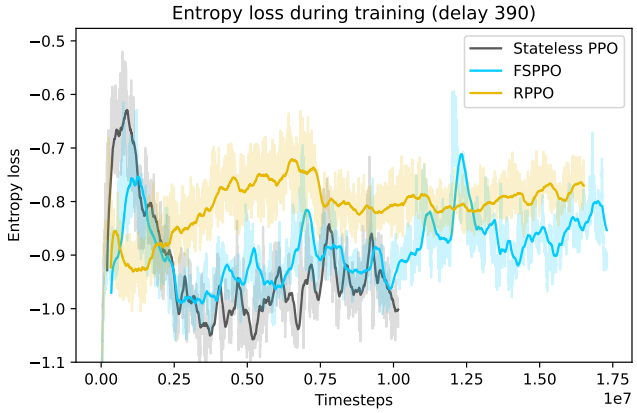

Fig. 27. Policy entropy during training for stateless PPO, FSPPO and RPPO in the delay scenario at delay 30 on tier 3

defining those axes cleanly is itself part of the work.

d) Meaningful ordering over arbitrary slots: The distance-sorting change described in Iterative Design and Artifact Development turned out to carry a principle beyond SYNGrid that any benchmark whose observation is a flat vector of symbolic entities can adhere to: ordering entities along an axis the task cares about, rather than an arbitrary slot order, frees the capacity the agent would otherwise spend learning slot mappings that carry no task meaning. In benchmarks like SYNGrid, where the agent is one entity among the others, distance to the agent is a natural axis to order by.

e) One-hot versus scalar trade-off: The move from one-hot to a scalar identity, described in Iterative Design and Artifact Development, exposed a scaling property worth carrying forward. One-hot was clean at low entity counts, but its width grew with the number of orbs in the pool as max tier increased. So in benchmarks built to scale that count, or those with many entities to begin with, the observation bloats as the task gets harder. A single scalar identity sidesteps this since it only produces one number, no matter how many entities exist. This problem isn't novel, MiniHack built on NetHack's roughly six thousand entity types represents each as a single integer id for the same reason [11]. The cost worth keeping in mind is that a scalar carries a magnitude, not just an identity, and that might leak ordinal information. In SYNGrid's tier scenario, where the encoding increases with tier and the task is to consume in ascending order, the scalar may hand a stateless agent a shortcut. Whether a categorical encoding that discards magnitude would remove this without sacrificing the fixed width is left to future work.

D. Broader Implications

Across the three scenarios, the harder challenge was not distinguishing architectures, but designing a signal clean enough to induce learning at all. As the newborn analogy in the Credit Assignment section suggests: we would not expect a child to discover the concept of ordered sequences from scratch with no structured feedback in a reasonable timeframe. Tabula rasa agents face the same compounded difficulty: learning

Fig. 28. RPPO's evaluation path in the spatial POMDP scenario, returning to the tier 3 orb after a prior encounter earlier in the episode

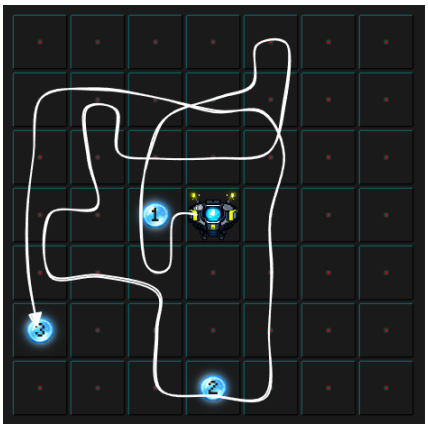

Fig. 29. FSPPO's search pattern on a similar grid — showing undirected search with no return to previously encountered orbs

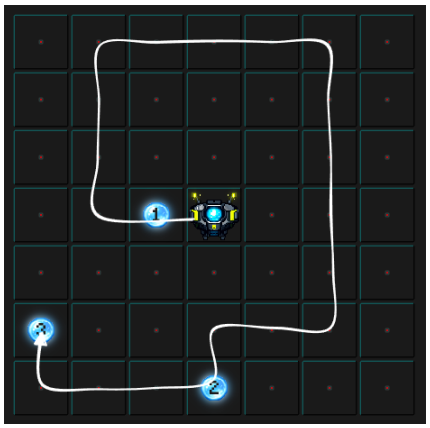

Fig. 30. RPPO's search pattern on the same grid as FSPPO. No previously encountered orbs, showing a broad circular sweep pattern

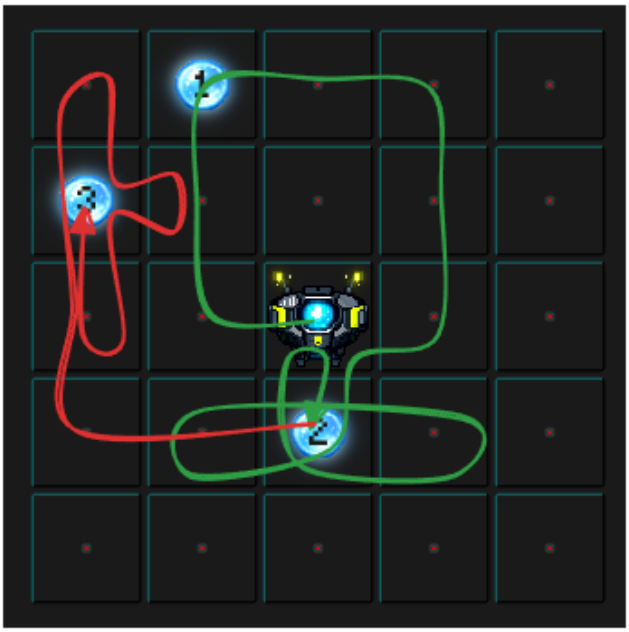

Fig. 31. RPPO evaluation path in the delay scenario, showing approximate homing behavior toward the last known position of the next orb in the consumption sequence

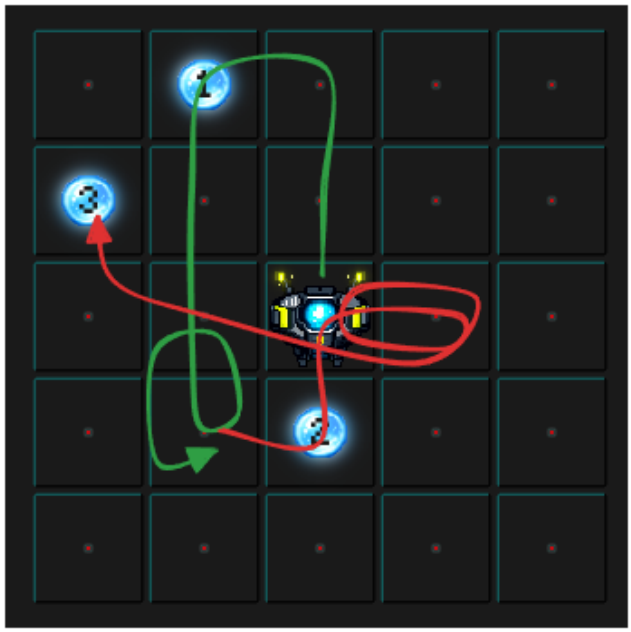

Fig. 32. Stateless PPO evaluation path in the delay scenario, showing looping behavior in place while waiting for the next orb to become available

what sequential dependency means and learning this specific sequence simultaneously. Perhaps reward maximization alone is not enough to solve certain tasks, and something more structured is needed from the ground up.

In the end, perhaps Pignatelli et al. were right in what they touched upon in their discussion [3, sec. 8.2] — that agents without priors inherently struggle just because they have to learn even elementary associations from scratch. AXIOM from VERSES AI, an active inference agent, demonstrates that agents equipped with structured priors about object dynamics sidestep much of this struggle, learning in thousands of steps rather than millions [13]. A question they bring up in their conclusion is whether those priors can be learned automatically from experience rather than being hand crafted for each scenario and seems like an interesting direction for the future. Furthermore, AXIOM is built on the free energy principle which frames intelligent behavior not as reward maximization but as surprise minimization. The idea is that the agent (biological or artificial) acts to keep the world predictable rather than purely to gain. This connects to a broader finding in neuroscience: perceptual decisions are biased not only by the associated reward but also by the cost to act [14]. Taken together, both suggest that biological intelligence is

shaped not by reward alone but by multiple overlapping drives: predictability, effort, and outcome. Reward-maximizing agents usually carry only the one. Whether that narrowness is a limitation worth addressing, or simply a different design point, is left open here.

IX. Ethical and Sustainability Considerations

While SYNGrid is not intended to compete in speed with already lightweight benchmarks like MiniGrid or NetHack, its focus on scenario complexity with a smaller action space can reduce training steps and compute compared to more complex environments such as VizDoom or Starcraft [4], [5], while still preserving meaningful task complexity. This efficiency lowers energy consumption and resource use when evaluating agents, and could support more sustainable AI research practices. However, whether SYNGrid truly possesses these capabilities remains to be seen, as this is not part of the evaluation as much as a preliminary claim, with the benchmark's simplicity serving as the main justification rather than its ability to match the full complexity of more advanced environments.

X. Conclusions

This thesis set out to evaluate SYNGrid as a lightweight benchmark, asking to what extent it can isolate TCA across agent architectures, and whether its configuration-driven design supports flexible scenario engineering without code changes. The aim was not to solve the CAP or train optimal agents, but to assess the benchmark's diagnostic utility through them. What emerged was a benchmark framework that, even in prototype form, produces measurable and interpretable differentiation across agent architectures through configuration alone. Just as consistently, the configuration changes produced outcomes matching the intentions behind them across all three comparisons. In the process it became clear that designing a clean isolated signal is harder than it appears as sparse reward and sequential dependency compound each other, which itself point toward learned priors as a direction worth pursuing. Taken together, SYNGrid offers a starting point — a configurable testbed that could grow alongside the field, both for stress-testing existing architectures and probing whatever directions emerge next.

References

[1] M. Minsky, "Steps toward Artificial Intelligence," *Proceedings of the IRE*, vol. 49, no. 1, pp. 8–30, Jan. 1961. [Online]. Available: https://ieeexplore.ieee.org/document/4066245/

[2] C.-C. Hung, T. Lillicrap, J. Abramson, Y. Wu, M. Mirza, F. Carnevale, A. Ahuja, and G. Wayne, "Optimizing agent behavior over long time scales by transporting value," *Nature Communications*, vol. 10, no. 1, p. 5223, Nov. 2019. [Online]. Available: https://www.nature.com/articles/s41467-019-13073-w

[3] E. Pignatelli, J. Ferret, M. Geist, T. Mesnard, H. v. Hasselt, O. Pietquin, and L. Toni, "A Survey of Temporal Credit Assignment in Deep Reinforcement Learning," Jul. 2024. [Online]. Available: http://arxiv.org/abs/2312.01072

[4] A. Khan and A. Aqeel, "Benchmarking reinforcement learning algorithms in first-person shooter games using VizDoom," *Entertainment Computing*, vol. 55, p. 101031, Sep. 2025. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S1875952125001119

[5] Y. Li, Y. Wang, and Y. Zhou, "Multiagent Deep Reinforcement Learning Algorithms in StarCraft II: A Review," *IEEE Access*, vol. PP, pp. 1–1, 01 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10752399

[6] R. S. Sutton and A. Barto, *Reinforcement learning: an introduction*, 2nd ed., ser. Adaptive computation and machine learning. Cambridge, Massachusetts London, England: The MIT Press, 2020. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," Aug. 2017. [Online]. Available: http://arxiv.org/abs/1707.06347

[8] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: https://www.nature.com/articles/nature14236

[9] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[10] M. Chevalier-Boisvert, B. Dai, M. Towers, R. de Lazcano, L. Willems, S. Lahlou, S. Pal, P. S. Castro, and J. Terry, "Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/e8916198466e8ef218a2185a491b49fa-Abstract-Datasets_and_Benchmarks.html

[11] M. Samvelyan, R. Kirk, V. Kurin, J. Parker-Holder, M. Jiang, E. Hambro, F. Petroni, H. Küttler, E. Grefenstette, and T. Rocktäschel, "Minihack the planet: A sandbox for open-ended reinforcement learning research," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, J. Vanschoren and S. Yeung, Eds., 2021. [Online]. Available: https://openreview.net/forum?id=skFwlyefkWJ

[12] A. Hevner and S. Chatterjee, *Design Research in Information Systems: Theory and Practice*, ser. Integrated Series in Information Systems. Boston, MA: Springer US, 2010, vol. 22. [Online]. Available: https://link.springer.com/10.1007/978-1-4419-5653-8

[13] C. Heins, T. V. d. Maele, A. Tschantz, H. Linander, D. Markovic, T. Salvatori, C. Pezzato, O. Catal, R. Wei, M. Koudahl, M. Perin, K. Friston, T. Verbelen, and C. Buckley, "AXIOM: Learning to Play Games in Minutes with Expanding Object-Centric Models," May 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2505.24784

[14] N. Hagura, P. Haggard, and J. Diedrichsen, "Perceptual decisions are biased by the cost to act," *eLife*, vol. 6, p. e18422, Feb. 2017. [Online]. Available: https://doi.org/10.7554/eLife.18422

APPENDIX

APPENDIX A: TIME PLAN

Fig. 33. Initial Roadmap

Fig. 34. Final Roadmap